PEMROGRAMAN BERORIENTASI OBJEK 2026

Enkapsulasi

Membuat objek menjaga datanya sendiri: data yang tidak masuk akal ditolak sejak di pintu masuk

PERTEMUAN

04

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Menjelaskan masalah yang muncul jika data objek bisa diubah sembarangan dari luar kelas.
2. Menjelaskan arti **enkapsulasi** dan ***information hiding*** dengan analogi sehari-hari.
3. Membedakan `public`, `protected`, *default*, dan `private` di Java, serta konvensi `_nama` dan `__nama` di Python.
4. Merancang ***getter***, ***setter***, dan atribut ***read-only*** yang memvalidasi data, di *setter* maupun di konstruktor.
5. Memilih *method* “penjaga aturan” alih-alih *setter* dan menjelaskan kapan *getter*/*setter* tidak diperlukan.
6. Menerapkan enkapsulasi pada studi kasus KedaiKita.

Mengingat Kembali



Dari pertemuan sebelumnya

- Ada **atribut objek** (milik tiap objek) dan **atribut kelas** (milik bersama, misalnya penghitung `jumlah_menu`).
- Konstruktor bisa dibuat luwes: nilai bawaan `stok=0` di Python, *overloading* dengan `this(...)` di Java.
- `__str__` (Python) dan `toString()` (Java) membuat objek bisa langsung dicetak dengan rapi.
- Variabel menyimpan **referensi**. Siapa pun yang memegang referensi sebuah objek bisa mengubah datanya.

Agenda Hari Ini

1. Mengapa data perlu dijaga?
2. *Access modifier*: siapa boleh masuk?
3. *Getter*, *setter*, dan validasi
4. *Read-only* dan *method* penjaga aturan
5. Studi kasus KedaiKita

BAGIAN 1

1

Mengapa Data Perlu Dijaga?

Dari harga kopi yang minus menuju enkapsulasi.

Cerita Pembuka: Harga Kopi Jadi Minus

Data yang rusak	Akibatnya
Harga kopi minus Rp18.000	Pelanggan yang memesan 2 kopi justru “dibayar” Rp36.000
Stok kopi minus 5	Kasir masih menerima pesanan untuk menu yang sebenarnya sudah habis
Poin Budi 1.000.000	Budi bisa menukar poinnya menjadi potongan Rp1 miliar (1 poin = Rp1.000)

Program tetap berjalan tanpa protes. Masalahnya ada pada **desain kelas**: kelas tidak punya cara untuk berkata “tidak”.

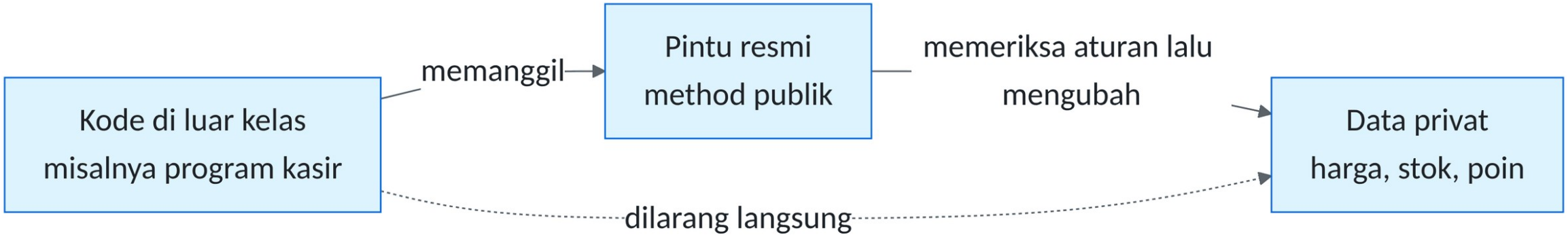
Enkapsulasi: Membungkus dan Menjaga

Enkapsulasi (*encapsulation*)

Prinsip membungkus data dan *method* yang mengolahnya ke dalam satu kelas, lalu **membatasi akses langsung** ke data tersebut. Pihak luar hanya berinteraksi lewat “pintu resmi”, yaitu *method* yang disediakan kelas.

- **Membungkus**: data dan *method* disatukan dalam satu kelas, seperti serbuk obat dalam kapsul.
- **Menyembunyikan** (*information hiding*): pihak luar cukup tahu **apa** yang bisa dilakukan, bukan **bagaimana**.
- Mesin ATM: brankas tertutup rapat, yang terlihat hanya menu cek saldo, tarik tunai, dan transfer.

Pintu Resmi dan Data Privat



Wajah depan (*public interface*) boleh dipakai semua kode; dapurnya (data privat) hanya untuk kode di dalam kelas itu.

BAGIAN 2

2

Access Modifier: Siapa Boleh Masuk?

Aturan tentang siapa yang boleh menyentuh atribut dan *method*.

Empat Tingkat Akses di Java

Access modifier	Kelas lain di paket yang sama	Kelas anak di paket lain	Kelas lain di paket lain
`public`	boleh	boleh	boleh
`protected`	boleh	boleh	tidak
default (tanpa kata kunci)	boleh	tidak	tidak
`private`	tidak	tidak	tidak

Kelas itu sendiri selalu boleh. Pola paling umum: atribut dibuat `private`, *method* untuk umum dibuat `public`.

Python: Kesepakatan lewat Nama

Penulisan nama	Contoh	Artinya	Dijaga Python?
Tanpa garis bawah	`nama`	Publik, boleh dipakai siapa saja	-
Satu garis bawah di depan	`_harga`	“Urusan internal, jangan disentuh dari luar kelas”	Tidak, hanya kesepakatan
Dua garis bawah di depan	`__pin`	Namanya diganti menjadi `_NamaKelas__pin` (*name mangling*)	Hanya namanya yang diganti

Jangan tertukar dengan `__init__` atau `__str__`: nama yang diapit dua garis bawah di depan **dan** belakang adalah *method* khusus.

Name Mangling: Diganti, Bukan Dikunci

`self.__pin` disimpan dengan nama lain. `brankas.__pin` tidak ditemukan, tetapi nama barunya tetap bisa dibaca.

Python python/contoh/contoh02_name_mangling.py

```python
class Brankas:
    def __init__(self, pemilik, pin):
        self._pemilik = pemilik
        self.__pin = pin
# ... di dalam main()
brankas = Brankas("Budi", "246810")
# ... (dilewati)
print(f"Ada atribut '__pin'?  : {ya_tidak(hasattr(brankas, '__pin'))}")
print(f"Isi objek sebenarnya  : {vars(brankas)}")
# ... (dilewati)
print(f"brankas._Brankas__pin : {brankas._Brankas__pin}")
```

Khusus Python

Dua garis bawah di depan nama memicu *name mangling*. Java tidak punya padanannya.

Output

```
Ada atribut '__pin'?  : tidak
Isi objek sebenarnya  : {'_pemilik': 'Budi', '_Brankas__pin': '246810'}
brankas._Brankas__pin : 246810
```

BAGIAN 3

3

Getter, *Setter*, dan Validasi

Pintu baca, pintu ubah, dan cara menolak data yang tidak masuk akal.

Getter dan *Setter*: Pintu Baca dan Pintu Ubah

Kebutuhan	Python (*property*)	Java (*getter/setter*)
Membaca harga	`kopi.harga`	`kopi.getHarga()`
Mengubah harga	`kopi.harga = 20000`	`kopi.setHarga(20000)`
Membaca data ya/tidak	`parkiran.penuh`	`parkiran.isPenuh()`
Di balik layar	*method* bertanda `@property` dan `@harga.setter`	*method* biasa bernama `getX`, `setX`, atau `isX`

Getter tidak harus mengembalikan atribut apa adanya: ia boleh menghitung atau merapikan, misalnya sisa tempat parkir.

Property di Python

`@property` membuat *getter*, `@volume.setter` membuat *setter*. Volume di luar 0—100 ditolak dengan `raise`.

Python python/contoh/contoh03_property.py

```python
class Televisi:
    def __init__(self, merek):
        self._merek = merek
        self._volume = 10
    # ... (dilewati)
    @property
    def volume(self):
        return self._volume

    @volume.setter
    def volume(self, nilai):
        if nilai < 0 or nilai > 100:
            raise ValueError(f"Volume harus 0-100, bukan {nilai}")
        self._volume = nilai

# ... di dalam main()
tv = Televisi("Polytron")
tv.volume = 25
```

Khusus Python

Property dipakai seperti atribut biasa. `tv.volume = 25` diam-diam memanggil *setter*.

Getter dan *Setter* di Java

Field `private`, lalu *method* `public` `getVolume()` dan `setVolume()`. Nilai tidak valid ditolak dengan `throw`.

Java

java/contoh/Contoh03GetterSetter.java

```java
class Televisi {
    private String merek;
    private int volume;
    // ... (dilewati)
    public int getVolume() {
        return volume;
    }

    public void setVolume(int volume) {
        if (volume < 0 || volume > 100) {
            throw new IllegalArgumentException("Volume harus 0-100...");
        }
        this.volume = volume;
    }
}
// ... di dalam main
Televisi tv = new Televisi("Polytron");
tv.setVolume(25);
```

Khusus Java

Getter dan *setter* di Java adalah *method* biasa bernama `getX` dan `setX`, dipanggil dengan tanda kurung.

Dua Pintu: *Setter* dan Konstruktor

Konstruktor memanggil *setter*, jadi aturan cukup ditulis sekali dan berlaku di kedua pintu.

Python python/contoh/contoh04_validasi_konstruktor.py

```python
class Mahasiswa:
    def __init__(self, nama, semester):
        self.nama = nama
        self.semester = semester
    # ... (dilewati)
    @nama.setter
    def nama(self, nilai):
        if nilai.strip() == "":
            raise ValueError("Nama tidak boleh kosong")
        self._nama = nilai
```

Java java/contoh/Contoh04ValidasiKonstruktor.java

```java
class Mahasiswa {
    // ... (dilewati)
    public Mahasiswa(String nama, int semester) {
        setNama(nama);
        setSemester(semester);
    }
    // ... (dilewati)
    public void setNama(String nama) {
        if (nama.isBlank()) {
            // ... lempar IllegalArgumentException
        }
        this.nama = nama;
    }
}
```

BAGIAN 4

4

Read-Only dan *Method* Penjaga Aturan

Data yang tidak boleh berubah dan perubahan yang punya makna.

Atribut *Read-Only*: Boleh Dilihat, Tidak Boleh Diubah

Atribut *read-only* (hanya baca)

Atribut yang bisa **dibaca** dari luar kelas, tetapi **tidak bisa diubah** dari luar kelas. Caranya: sediakan *getter*, tetapi **jangan** sediakan *setter*.

- Python: *property* tanpa *setter*. Mencoba mengubahnya memicu `AttributeError` saat program berjalan.
- Java: *getter* tanpa *setter*, jadi mengubahnya ditolak saat kompilasi. `final` membuatnya hanya bisa diisi sekali.
- Satu-satunya kesempatan memeriksanya ada di konstruktor, misalnya NIK harus 16 digit.

Method Penjaga Aturan

Pembeda	*Setter* stok (dihindari)	*Method* penjaga aturan
Perintahnya	“Ubah stok menjadi 15”	“Kurangi stok sebanyak 5”
Kelas tahu alasannya?	Tidak	Ya, menu terjual
Bisa menolak penjualan yang melebihi stok?	Sulit	Mudah
Mudah ditambah pencatatan di kemudian hari?	Sulit	Mudah, cukup di dalam *method* itu

Contoh: `tambah_stok(jumlah)` untuk restok dan `kurangi_stok(jumlah)` untuk penjualan (Java: `tambahStok`, `kurangiStok`).

Kapan Tidak Perlu *Getter* dan *Setter*?

- Perlu **dibaca** dari luar? Jika tidak, jangan beri *getter*. Contoh: *password*.
- Perlu **diubah** dari luar? Jika tidak, jadikan *read-only*. Contoh: kode menu.
- Perubahannya punya **makna khusus** (terjual, restok, setor)? Buat *method* penjaga aturan.
- Jika *setter* memang tepat, pastikan *setter* itu **memeriksa** nilai barunya.

Penting

Tell, don't ask: **suruh** objek mengerjakan tugasnya, misalnya `kopi.kurangi_stok(2)`. Jangan tanya datanya lalu memutuskan sendiri di luar kelas.

STUDI KASUS

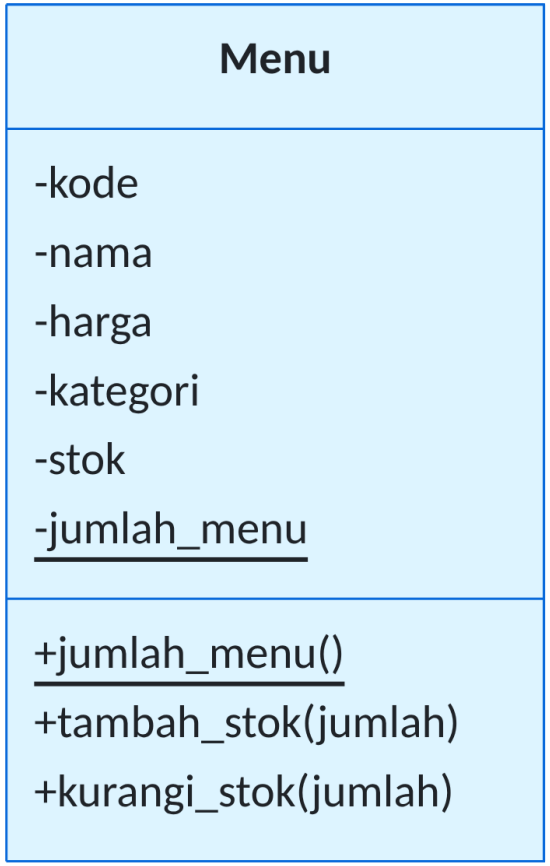


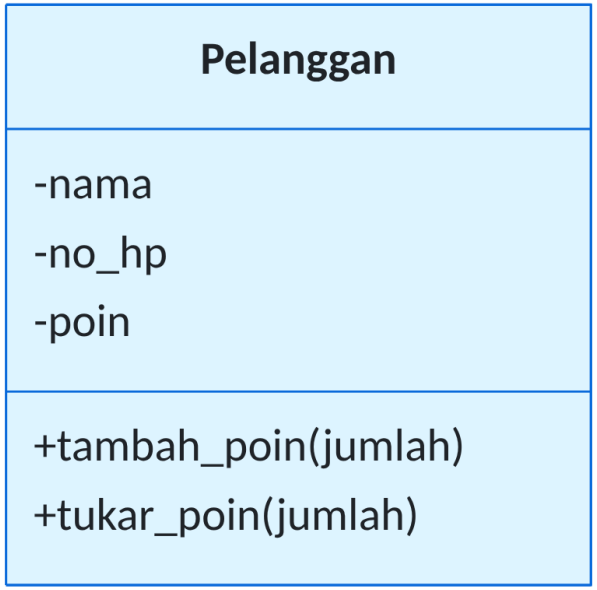
Studi Kasus KedaiKita

Menu dan pelanggan yang menjaga datanya sendiri.

Rancangan KedaiKita Pertemuan 04

Studi kasus KedaiKita





Tanda - berarti privat, + berarti publik, dan anggota yang digarisbawahi adalah milik kelas.

Aturan yang Kini Dijaga Kelas

Studi kasus KedaiKita

Aturan	Dijaga oleh	Jika dilanggar
Harga harus lebih dari 0	*Setter* harga (juga dipakai konstruktor)	Ditolak: “Harga harus lebih dari 0”
Stok awal tidak boleh negatif	Konstruktor	Ditolak: “Stok tidak boleh negatif”
Penjualan tidak boleh melebihi stok	`kurangi_stok`	Ditolak: “Stok … tidak cukup (sisa …)”
Kode, nama, dan kategori tidak berubah	Tidak ada *setter* (*read-only*)	Python: `AttributeError`; Java: *error* kompilasi
Poin yang ditukar tidak boleh melebihi poin yang dimiliki	`tukar_poin`	Ditolak: “Poin tidak cukup (sisa …)”
Penghitung hanya naik jika menu berhasil dibuat	Konstruktor (langkah terakhir)	Menu yang gagal dibuat tidak terhitung

Stok Hanya Berubah lewat *Method* Resmi

Studi kasus KedaiKita

Terjual 5, restok 10, lalu mencoba menjual 30 sekaligus. Objek kopi sendiri yang menolak.

Python python/studi-kasus/kedaikita.py

```python
kopi.kurangi_stok(5)
# ... (dilewati)
kopi.tambah_stok(10)
# ... (dilewati)
try:
    kopi.kurangi_stok(30)
except ValueError as e:
    print(f"Gagal menjual: {e}")
```

Java java/studi-kasus/KedaiKita.java

```java
kopi.kurangiStok(5);
// ... (dilewati)
kopi.tambahStok(10);
// ... (dilewati)
try {
    kopi.kurangiStok(30);
} catch (IllegalArgumentException e) {
    System.out.println("Gagal menjual: " + e.getMessage());
}
```

Output

```
Gagal menjual: Stok Kopi Susu Gula Aren tidak cukup (sisa 25)
```

Miskonsepsi yang Sering Terjadi

Anggapan keliru	Yang benar
“Enkapsulasi artinya semua atribut dibuat privat, lalu semuanya diberi *getter* dan *setter*.”	*Setter* yang menerima nilai apa saja sama saja dengan atribut publik. Beri *getter*/*setter* hanya jika perlu dan pakai *method* penjaga aturan untuk perubahan bermakna.
“Atribut privat membuat data aman dari peretas.”	*Access modifier* mencegah kesalahan pemakaian di dalam program, bukan serangan dari luar. Di Python, garis bawah hanyalah kesepakatan; *name mangling* hanya mengganti nama.
“*Getter* harus mengembalikan atribut apa adanya.”	*Getter* boleh menghitung (sisa tempat parkir) atau merapikan tampilan (nomor antrean `A007`). Yang penting, pemakai mendapat informasi yang benar.

Kuis Cepat

Di mesin ATM, kamu tidak bisa membuka brankas berisi uang. Kamu hanya bisa memakai menu “tarik tunai” yang memeriksa PIN, saldo, dan batas harian sebelum uang keluar. Dalam analogi enkapsulasi, menu itu setara dengan …

A. atribut privat

B. nama kelas

C. *method* publik yang menjaga aturan

D. paket (*package*)

Kuis Cepat: Jawaban

Di mesin ATM, kamu tidak bisa membuka brankas berisi uang. Kamu hanya bisa memakai menu “tarik tunai” yang memeriksa PIN, saldo, dan batas harian sebelum uang keluar. Dalam analogi enkapsulasi, menu itu setara dengan …

A atribut privat

B nama kelas

✓ *method* publik yang menjaga aturan

D paket (*package*)

Jawaban: C

Menu “tarik tunai” adalah pintu resmi yang boleh dipakai siapa saja, tetapi selalu memeriksa aturan sebelum mengubah data (saldo). Itulah peran *method* publik penjaga aturan. Uang di dalam brankas justru setara dengan **atribut privat** (pilihan A), yaitu data yang tidak boleh disentuh langsung.

Kuis Cepat

Ada dua cara mengurangi stok kopi sebanyak 2 porsi. Mana yang mengikuti prinsip ***tell, don't ask***?

```
Cara P: kopi.kurangi_stok(2)        (Java: kopi.kurangiStok(2))
Cara Q: jika stok kopi >= 2, ubah stok kopi menjadi stok - 2
```

A Cara P karena objek kopi sendiri yang memeriksa aturan dan menolak jika stok tidak cukup

B Cara Q karena pemanggil memeriksa aturan sendiri sebelum mengubah data

C Keduanya sama baiknya, asalkan hasil akhirnya benar

D Tidak ada karena prinsip itu melarang memanggil *method* milik objek lain

Kuis Cepat: Jawaban

Ada dua cara mengurangi stok kopi sebanyak 2 porsi. Mana yang mengikuti prinsip ***tell, don't ask***?

✓ Cara P karena objek kopi sendiri yang memeriksa aturan dan menolak jika stok tidak cukup

B Cara Q karena pemanggil memeriksa aturan sendiri sebelum mengubah data

C Keduanya sama baiknya, asalkan hasil akhirnya benar

D Tidak ada karena prinsip itu melarang memanggil *method* milik objek lain

Jawaban: A

Tell, don't ask berarti **menyuruh** objek mengerjakan tugasnya, bukan **menanyakan** datanya lalu memutuskan di luar. Pada cara Q, aturan "stok tidak boleh minus" ditulis di luar kelas sehingga harus diulang di setiap tempat dan mudah terlupa. Cara Q juga butuh *setter* stok yang membuka pintu bagi data rusak.

Ringkasan

- **Enkapsulasi** membungkus data dan *method* dalam satu kelas, lalu membatasi akses langsung lewat **pintu resmi**.
- Java: `public`, `protected`, *default*, `private` dijaga *compiler*. Python: `_nama` hanya kesepakatan.
- `__nama` di Python memicu *name mangling*: namanya diganti, bukan dikunci. Itu **bukan** fitur keamanan.
- ***Getter*** membaca, ***setter*** mengubah sambil **memvalidasi**: *property* di Python, `getX`/`setX` di Java.
- Validasi wajib di *setter* **dan** konstruktor; data tidak valid ditolak dengan ***exception***.
- Atribut ***read-only*** punya *getter* tanpa *setter*; perubahan bermakna memakai *method* **penjaga aturan**.
- Jangan membuat *getter* dan *setter* secara otomatis. Ingat prinsip ***tell, don't ask***.

Langkah Berikutnya

- Jalankan contoh 01—07 dan studi kasus KedaiKita di jalur pilihanmu.
- Kerjakan latihan, lalu cek pemahamanmu dengan kuis 10 soal.

Buka bahan pertemuan ini di GitHub:

- Konsep
- Praktik Python
- Praktik Java
- Jembatan Python ↔ Java
- Latihan
- Kuis 10 soal

Pindai untuk membuka folder pertemuan ini di GitHub

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-04-Enkapsulasi